NAMAL UNIVERSITY MIANWALI

DEPARTMENT OF ELECTRICAL ENGINEERING+
EE-449L- Digital Image Processing (Elective)

Lab Semester Project

Project Report

Road Lane Detection Using Digital Image Processing

Submitted By

Muhammad Wajid Jamal (NUM-BSEE-2022-24)
Muhammad Usman Shah (NUM-BSEE-2022-28)

Submitted To:

Ms. Zulaikha Kiran
&
Ms. Aliza

Submission Date:
12 January 2026

Table of Contents

Abstract 3
Introduction: 4
Problem Domain and Input Images: 4
Problem Domain: 4
Types of Input Images Used: 4
Possible Algorithms for Road Lane Detection: 4
1. Edge-Based Detection: 5
2. Hough Transform Method: 5
3. Color-Based Segmentation 5
4. Detection using Deep Learning 5
Algorithm Implemented 6
Overview 6
Step-by-Step Working 6
Block Diagram: 7
Graphical User Interface (GUI): 8
Features of the GUI: 8
GUI Operation 8
Results and Discussion: 9
Analysis of color space (Hue, Saturation, Value) 9
Masking Performance: 10
Region of interest (ROI) Effectiveness: 11
Final Detection & Red Overlay 12
Example of Unsuccessful Detection 12
Assumptions and Limitations: 13
Assumptions 13
Limitations 14
Future Improvements 14
Conclusion: 14

Abstract

An intelligent transportation system requires road lane detection. It assists in determining the road lane markings using camera shots. This project applies the digital image processing methods in Python and OpenCV to detect a road lane. The image input is transformed into HSV color space by the algorithm and color-based segmentation is used to identify white and yellow lane markings. An ROI is chosen to target the road area and eliminate the unwanted background. The resultant picture shows the identified lane areas. These findings indicate that the technique is effective in bright daylight road scenes, though its efficiency declines in the poor lighting and shadow environments. Accuracy can be improved in the future and real-time detection is possible.

Introduction:

The detection of lanes on the road is a significant issue in the field of digital image processing. It has been applied in determining lane markings on roads using camera images. This is a technique that is common with intelligent transportation systems and driver assistance systems. The primary role of lane detection is to assist vehicles in the comprehension of road boundaries and in keeping the correct lanes position.

A system to detect road lanes is built in this project based on Python and OpenCV. The algorithm identifies white and yellow lane marks of images of roads. The Region of Interest (ROI) is chosen to concentrate on the road area only. Separating the lane pixels and the background is done by color-based segmentation in the HSV color space. The end product brings out identified lane areas. The present project illustrates that simple digital image processing can be utilized to address a practical issue in the area of transportation and automation.

Problem Domain and Input Images:

Problem Domain:

The problem field that has been chosen to be utilized in this project is road lane detection by means of digital image processing. Camera images are used to identify the lane markings on road surfaces by road lane detection. This technology plays a role in the contemporary transport systems. It finds extensive application in driver assistant systems and self-driving cars to enhance road safety and navigation.

The primary objective of this project is to identify any white and yellow lane markings in the image of the road and outline them. The detection process is also concerned with the road region and remains independent of the surrounding background, which includes buildings, trees, and vehicles, which are unnecessary.

Types of Input Images Used:

The images that have been used as an input in this project are front-view road images that have been taken using a camera. These pictures have visible white and yellow lane markings on the various kinds of roads. The pictures are captured under light during the day to enable enhanced view of the colors of the lanes.

Others have straight and transparent markings of the lane, and others have curves, shadows or lanes that are in poor conditions. These variations assist in the testing of the performance of the implemented algorithm in various conditions.

The pictures are inputted in the system via a graphical user interface and the algorithm interprets them to identify lane regions.

Possible Algorithms for Road Lane Detection:

The computer vision and digital image processing methods may be employed to detect road lanes markings using several algorithms. Below are some of the methods that are used frequently.

1. Edge-Based Detection:

Sobel or Canny is an operator that is used in edge-based detection to locate sharp changes of intensity in an image. As lane markings form powerful edges on roads, edge detection may identify lane markings.

Advantages:

- Simple and fast method
- Easy to implement

Disadvantages:

- Sensitive to noise
- In dim lights, there is reduced performance.

2. Hough Transform Method:

An image is used to detect straight lines by the Hough Transform. Once edge detection has been performed, the Hough Transform identifies line segments which indicate the lane markings.

Advantages:

- Good in straight lane lines detection.
- Performs excellently where lanes are clear.

Disadvantages:

- High computational cost
- Weaker when it is curvy.

3. Color-Based Segmentation

In this technique, the lane markings are identified with regard to color. Color filtering in HSR color space is used to extract white and yellow color. It is among the best and simplest methods of daylight image.

Advantages:

- Simple and efficient
- Excellence during a sunny day.

Disadvantages:

- Light responsive.
- Inability to perform in the dark.

4. Detection using Deep Learning

Convolutional neural networks (CNNs) are based on deep learning techniques that are used to identify lanes automatically in images.

Advantages:

- High accuracy
- Is able to identify curved and complex lanes.

Disadvantages:

- Large training datasets are required.
- Requires high level of computation.

Algorithm Implemented

Overview

This project is a road lane detection algorithm based on color and Python and OpenCV. The algorithm is concerned with the recognition of white and yellow lane markings of road pictures. The image is transformed into HSV color space in order to enhance accuracy. The detection is restricted to the road area by selecting a Region of Interest (ROI) manually. Lane regions that are final detected are marked in red color on the original image.

With the help of Tkinter, a graphical user interface (GUI) is created too that enables the user to upload an image and see the detection results without difficulty.

Step-by-Step Working

Step 1: Image Upload using GUI

Tkinter is used to develop a basic GUI. An upload button is selected by the user to choose an input road image.

Step 2: Reading the Image

The image chosen is read through OpenCV and transformed to RGB format to be displayed appropriately.

Step 3: RGB to HSV Conversion

The HSV color space is transformed to RGB image. The channels of Hue, Saturation and Value are separated.

Step 4: Creation of White Color mask.

The mask of white color is made by choosing the pixels which have low values of saturation and high values of brightness.

Step 5: Making of Yellow Color Mask.

The color mask is yellow because it is formed by choosing the pixels within the certain range of colors in terms of their hue, saturation, and brightness.

Step 6: Combine Color Masks

The white as well as the yellow mask is combined to identify all possible lane-colored pixels.

Step 7: Region of Interest Selection.

The road area is defined by the user by clicking on a polygon area on the image. This selection is made into a binary ROI mask.

Step 8: Apply ROI Mask

The overall color mask is multiplied with the ROI mask to eliminate background.

Step 9: Lane Detection and Highlighting.

The original image has spotted lane pixels denoted in red.

Step 10: Display Results

Processing stages, such as HSV channels, masks, ROI and final output are all shown in subplot windows. [1]

Block Diagram:

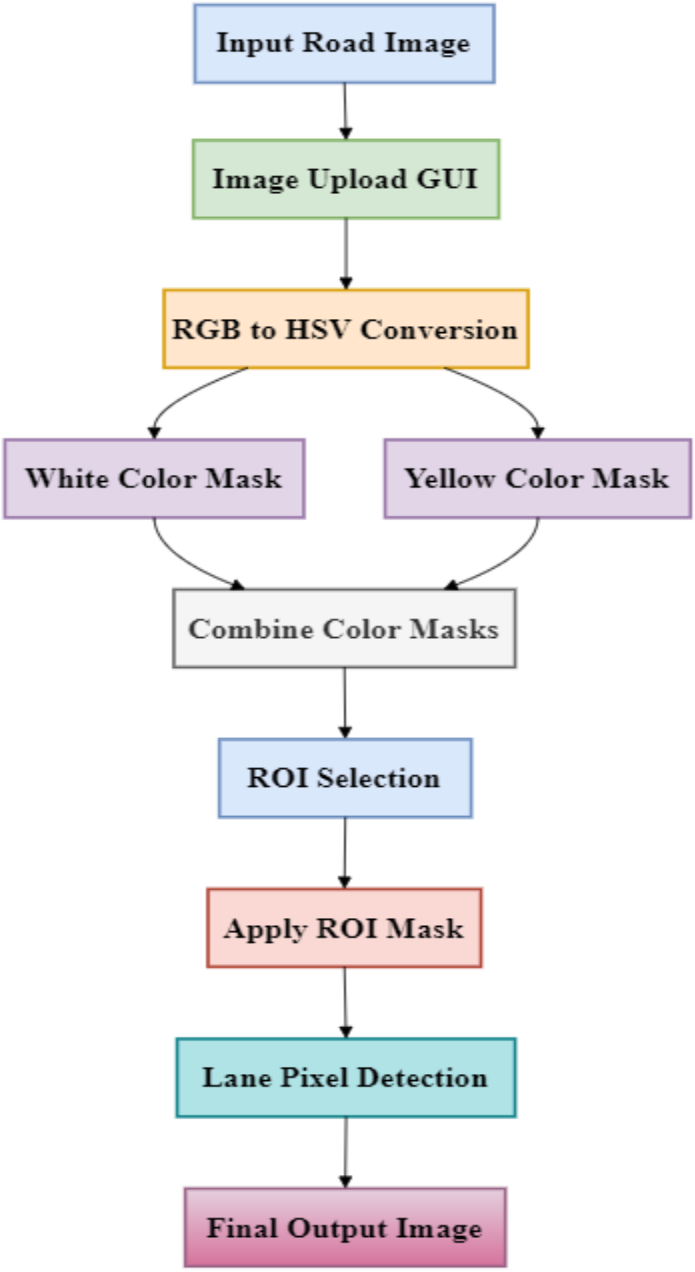


Figure 1 Block Diagram

Graphical User Interface (GUI):

The graphical user interface (GUI) is built as a basic one in Python by using Tkinter. The GUI enables the user to interrelate well with the lane detection system without programming anything.

Features of the GUI:

- **Image upload:** The user can retrieve any image of the road in the computer by clicking a button.
- **Automatic Processing:** The lane detection algorithm is conducted automatically after uploading the picture.
- **Result Display:** The final image of lanes detected is presented, and processing steps including HSV channels and masks were used in the middle of processing.
- **Easy to use:** The interface is user friendly and can be used by anyone.

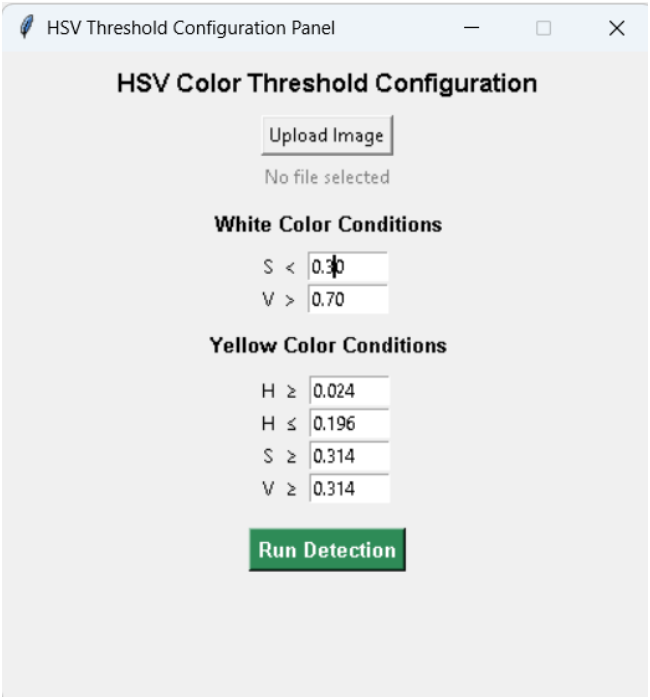


Figure 2:GUI Interface

GUI Operation

Run the Python GUI script.

- On the button Upload Image, click.
- Choose your road image as an input.

The image is processed by the program which displays:

- Original image
- HSV channels
- White and yellow masks
- Combined mask

- ROI mask

Final result using lanes identified in red.

The system is usable in testing various images with this GUI and visual confirmation of the lane detecting results. [2]

Results and Discussion:

The road lane detection algorithm that was implemented was put to test on several road images. The system is able to identify the white and yellow lane markings in clear images of the daytime. The HSV color space and Region of Interest (ROI), enhance the accuracy of the detection since it eliminates the unwanted backgrounds.

Original Image:

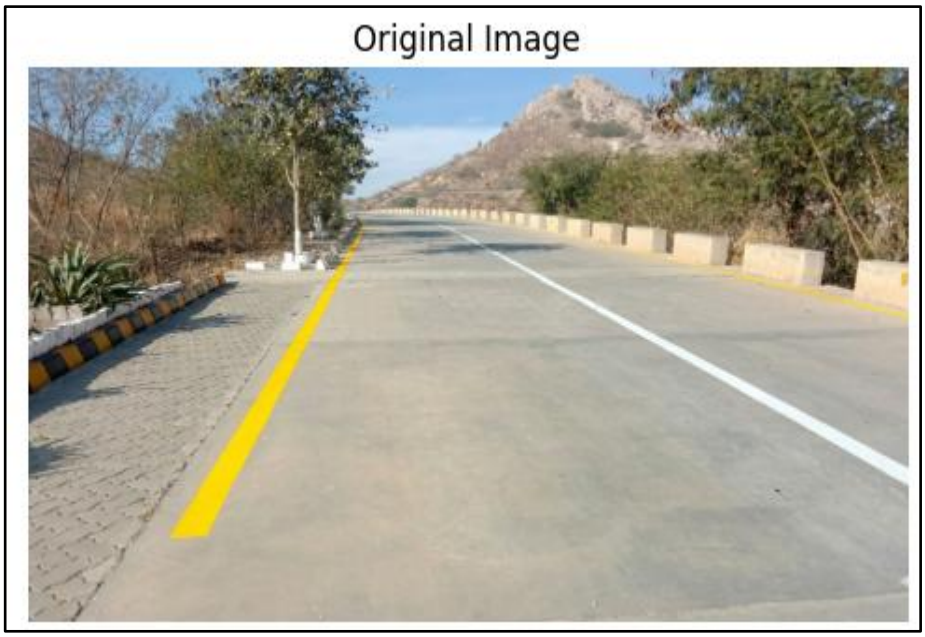


Figure 3: Original Image

Analysis of color space (Hue, Saturation, Value)

The algorithm processes the RGB image input to HSV color space that is important in identifying the lane markings.

- **Hue (H):** This differentiates between road surface and non-road in the form of trees or grass.
- **Saturation (S):** Assists in the identification of white lines as white areas lack saturation ($S < 0.3$).
- **Value (V):** Serves as a filter of brightness, showing only bright areas of the lane and disregarding the shadows.

Masking Performance:

- **White Mask:** White markings on roads are exactly recorded.
- **Yellow mask:** Identifies the yellow lanes, when they exist. A certain amount of noise might be present (e.g., grass objects, roadside objects).
- **Combined Mask:** This is a mixture of white and yellow lanes. Displayed all possible lane pixels, but can contain bright non-lane areas.

White Mask:

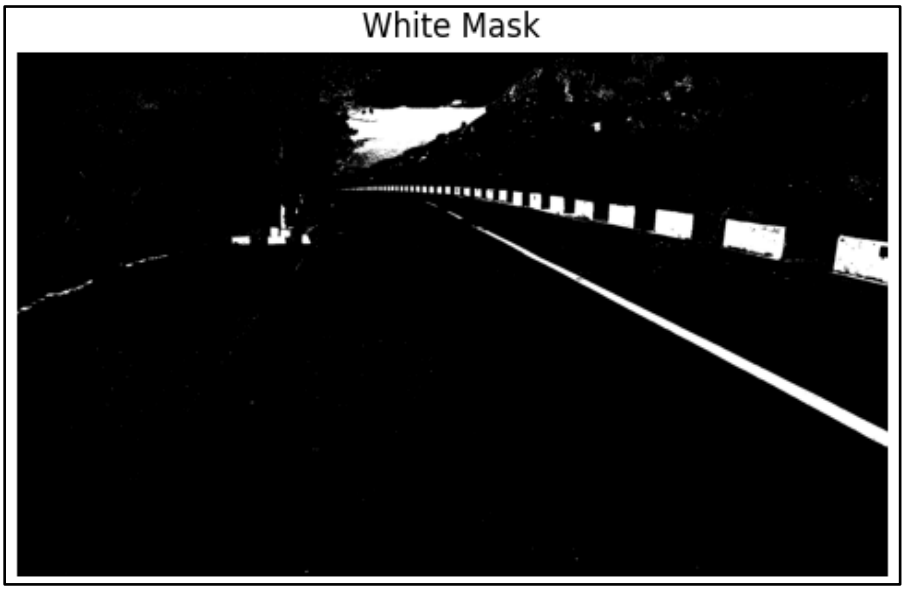


Figure 4 White Mask

Yellow Mask:

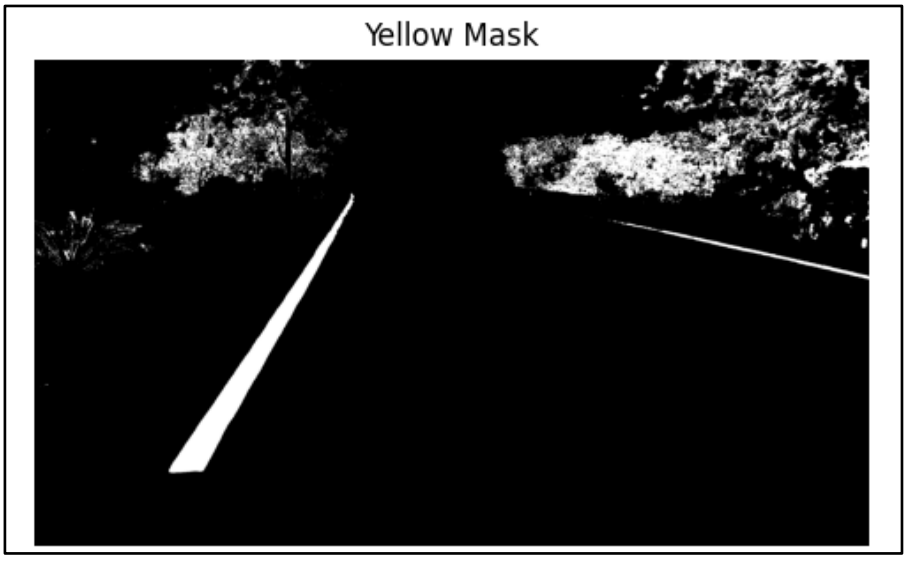


Figure 5 Yellow Mask

Combined Mask:

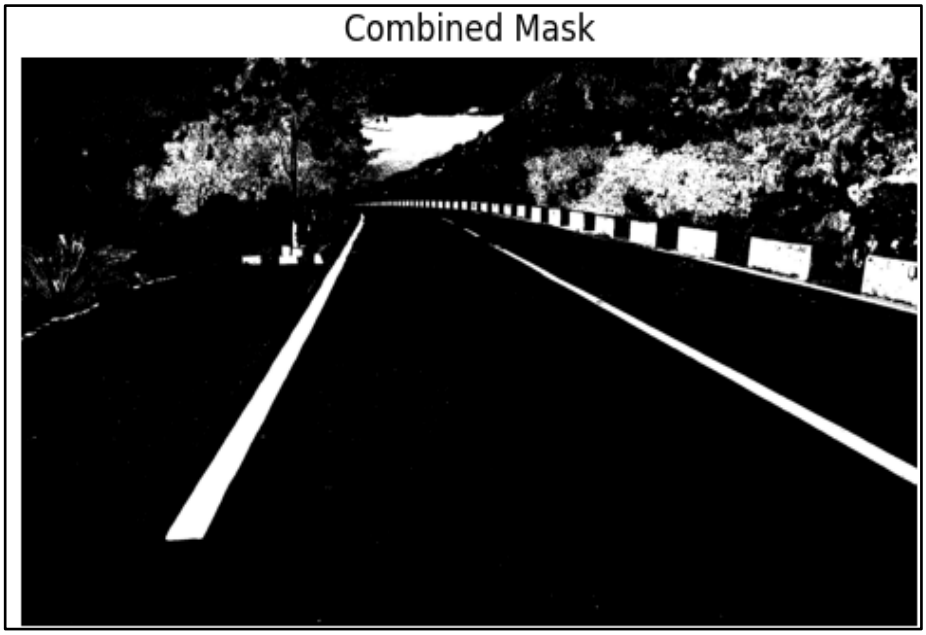


Figure 6 Combined Mask

Region of interest (ROI) Effectiveness:

The ROI Mask removes non-road features such as sky, trees and lanes.

- **Success:** Removes the majority of fake positives of the Combined Mask.
- **Output:** When the ROI masking step is implemented, the only pixels that are left are those associated with the road area.

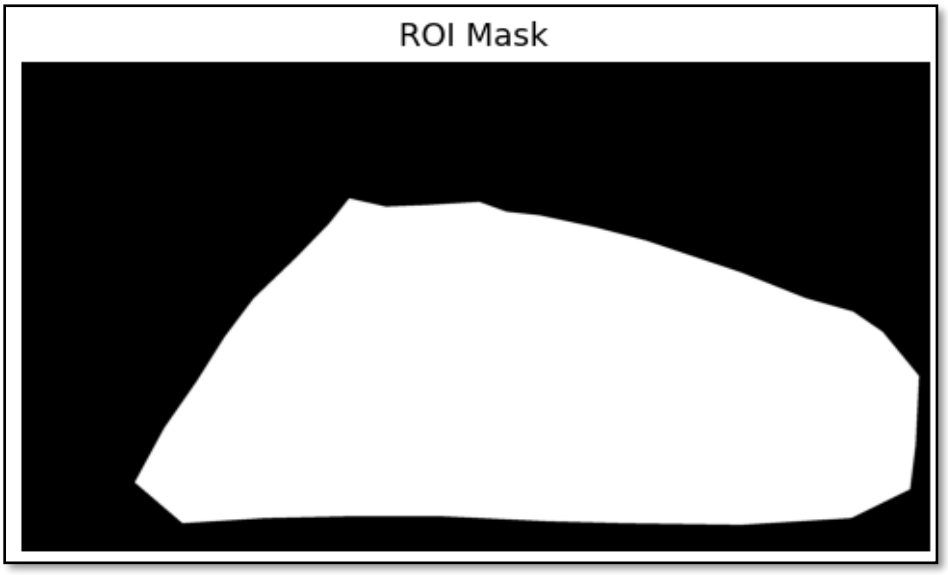


Figure 7 Region of Interest

Final Detection & Red Overlay

The end product identifies lane pixels with red color.

- **Precision:** Lane markings, such as dashed line, are easily identified.
- **Observation:** There will be some false detection on bright road barriers or shoulders in case they belong to the ROI.

Final Output:

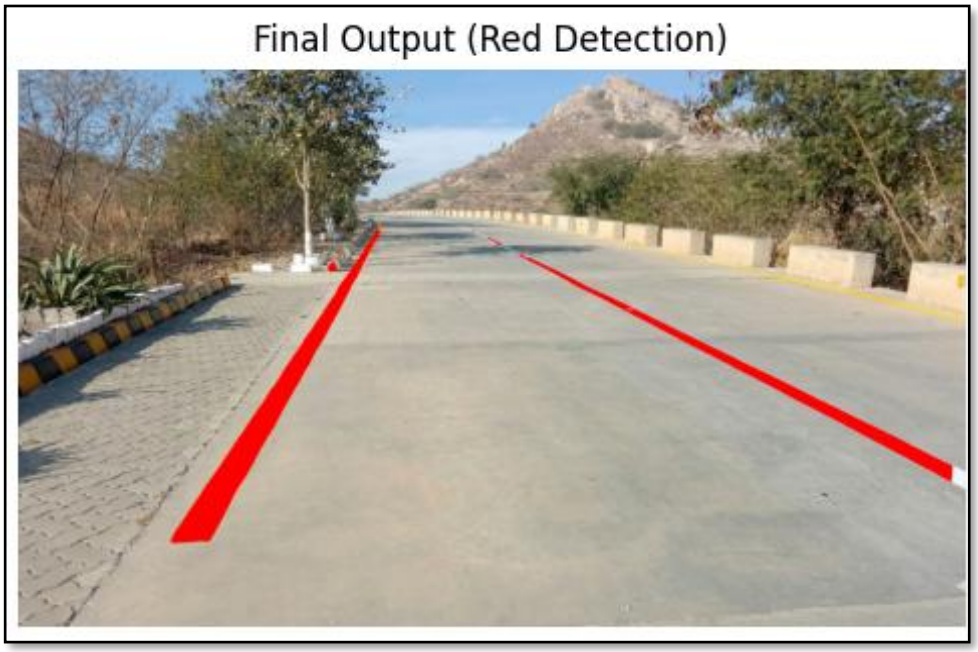


Figure 8 Red Detection Lane Output

Example of Unsuccessful Detection

Some images produce incorrect results due to challenging conditions:

- **Night-time or low-light image** – lane markings are not visible.
- **Shadowed roads** – bright areas under shadows can cause false detection.

Original Picture with shadow:



Figure 9 Original Picture with shadow

Final Output:



Figure 10 Final Output

Assumptions and Limitations:

Assumptions

- Lane markings are of white or yellow color.
- Images of the input are captured through a front facing camera on the car.
- Images of roads are obtained during the daytime.
- Lane detection does not pay attention to the surrounding environment but to the road itself that is covered by the ROI.

Limitations

- Each image needs to be selected through manual ROI.
- The performance is reduced when there is low lighting or during the night.
- Shadows or deteriorated lanes may result in improper detection.
- The algorithm identifies pixels on lanes and not straight lane lines.
- An occlusivity of curved lanes or roads can lower the accuracy of detection.

Future Improvements

Even though the current lane detection algorithm has been effective in daylight images, it can be enhanced to produce results in more complicated conditions and be more automated. The potential areas of improvement are:

Automatic ROI Selection: Eliminate the possibility of hand ROI selection by allowing the system to automatically locate the road area.

Canny Edge Detection Integration: The edges of the lanes are detected more accurately and nonbright objects are minimized as false alarms.

Hough Transform Lane Lines: Smooth lane representation: Build straight lines across the pixels found, and smooth out the lane representation.

Perspective (Bird's Eye) Transform: The road can be transformed into a top-down view to enhance the identification of the lane curvature and position of vehicles.

Strong to Lighting Effects: Improve the algorithm to operate in dark conditions and during the night, as well as in shaded parts.

Real-Time Video Processing: Build upon this to identify lanes in real-time video on cameras to be used in the automated cars.

Deep Learning Models: Lane detection models based on CNNs can be used to achieve the best accuracy, particularly on tricky roads.

Those advances will introduce more reliability, accuracy, and applicability of the system to real-life autonomous driving systems. [2]

Conclusion:

A road lane detection system was created in this project by use of Python and OpenCV. The algorithm identifies white and yellow lane markings with the help of HSV color segmentation and puts them on the original image in red. The use of manual ROI selection ensures that only the road area was processed leaving out the majority of the background noise.

The system presents itself as a good performer regarding clear daylight road image processing and shows efficiency of simple digital image processing techniques in lane recognition. Weaknesses like difficulty in performing in the dim light, shadows or dimmed lanes are observed and this can be improved in the future.

On the whole, this project manages to apply a practical lane detection algorithm and become an example of more sophisticated driver assistance and an autonomous vehicle system

References

[1] H. University. [Online]. Available: https://scholar.unhas.ac.id/en/publications/unstructured-road-detection-and-steering-assist-based-on-hsv-colo/?utm_source=chatgpt.com. [Accessed 12 01 2026].

[2] [Online]. Available: https://milestoneresearch.in/JOURNALS/index.php/IJHCI/article/view/92?utm_source=chatgpt.com. [Accessed 12 01 2026].

[3] I. Journal. [Online]. Available: https://www.ijraset.com/research-paper/lane-line-detection?utm_source=chatgpt.com. [Accessed 12 01 2026].

APPENDIX

Code:

```
import cv2
import numpy as np
from tkinter import *
from tkinter import filedialog
import matplotlib.pyplot as plt

file_path = None

# ------------------------------
# Upload Image
# ------------------------------
def upload_image():
    global file_path
    file_path = filedialog.askopenfilename(title="Select an image")
    path_label.config(text=file_path.split("/")[-1])

# ------------------------------
# Process Image
# ------------------------------
def process_image():
    if not file_path:
        print("No image selected")
        return

    img = cv2.imread(file_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    H = hsv[:, :, 0] / 255
    S = hsv[:, :, 1] / 255
    V = hsv[:, :, 2] / 255

    # Read GUI values
    white_s = float(whiteS_entry.get())
    white_v = float(whiteV_entry.get())

    h_min = float(yellowHmin_entry.get())
    h_max = float(yellowHmax_entry.get())
    s_min = float(yellowS_entry.get())
    v_min = float(yellowV_entry.get())

    # Masks
    whiteMask = (S < white_s) & (V > white_v)
```

```
yellowMask = (H >= h_min) & (H <= h_max) & (S >= s_min) & (V >= v_min)
colorMask = whiteMask | yellowMask

# ROI
plt.figure()
plt.imshow(img_rgb)
plt.title("Select ROI (Double-click to finish)")
roi_points = plt.ginput(n=-1, timeout=0)
plt.close()

if len(roi_points) < 3:
    print("ROI not selected properly")
    return

roi_points = np.array(roi_points, dtype=np.int32)
roi_mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.fillPoly(roi_mask, [roi_points], 1)
roi_mask = roi_mask.astype(bool)

finalMask = colorMask & roi_mask

finalOutput = img_rgb.copy()
finalOutput[finalMask] = [255, 0, 0]

# --------------------------------
# Plot results inside the function
# --------------------------------
plt.figure(figsize=(15, 10))

plt.subplot(3, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(whiteMask, cmap='gray')
plt.title("White Mask")
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(yellowMask, cmap='gray')
plt.title("Yellow Mask")
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(colorMask, cmap='gray')
```

```
    plt.title("Combined Mask")
    plt.axis("off")

    plt.subplot(3, 2, 5)
    plt.imshow(roi_mask, cmap='gray')
    plt.title("ROI Mask")
    plt.axis("off")

    plt.subplot(3, 2, 6)
    plt.imshow(finalOutput)
    plt.title("Final Output (Red Detection)")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

# ------------------------------
# GUI Layout
# ------------------------------
root = Tk()
root.title("HSV Threshold Configuration Panel")
root.geometry("420x420")
root.resizable(False, False)

Label(root, text="HSV Color Threshold Configuration",
      font=("Arial", 12, "bold")).pack(pady=8)

Button(root, text="Upload Image", command=upload_image).pack()
path_label = Label(root, text="No file selected", fg="gray")
path_label.pack(pady=3)

# ------------------------------
# White Mask Section
# ------------------------------
Label(root, text="White Color Conditions",
      font=("Arial", 10, "bold")).pack(pady=6)

frame_white = Frame(root)
frame_white.pack()

Label(frame_white, text="S").grid(row=0, column=0)
Label(frame_white, text="<").grid(row=0, column=1)
whiteS_entry = Entry(frame_white, width=8)
whiteS_entry.grid(row=0, column=2, padx=5)
whiteS_entry.insert(0, "0.30")
```

```
Label(frame_white, text="V").grid(row=1, column=0)
Label(frame_white, text=">").grid(row=1, column=1)
whiteV_entry = Entry(frame_white, width=8)
whiteV_entry.grid(row=1, column=2, padx=5)
whiteV_entry.insert(0, "0.70")

# ------------------------------
# Yellow Mask Section
# ------------------------------
Label(root, text="Yellow Color Conditions",
     font=("Arial", 10, "bold")).pack(pady=8)

frame_yellow = Frame(root)
frame_yellow.pack()

Label(frame_yellow, text="H").grid(row=0, column=0)
Label(frame_yellow, text="≥").grid(row=0, column=1)
yellowHmin_entry = Entry(frame_yellow, width=8)
yellowHmin_entry.grid(row=0, column=2, padx=5)
yellowHmin_entry.insert(0, "0.024")

Label(frame_yellow, text="H").grid(row=1, column=0)
Label(frame_yellow, text="≤").grid(row=1, column=1)
yellowHmax_entry = Entry(frame_yellow, width=8)
yellowHmax_entry.grid(row=1, column=2, padx=5)
yellowHmax_entry.insert(0, "0.196")

Label(frame_yellow, text="S").grid(row=2, column=0)
Label(frame_yellow, text="≥").grid(row=2, column=1)
yellowS_entry = Entry(frame_yellow, width=8)
yellowS_entry.grid(row=2, column=2, padx=5)
yellowS_entry.insert(0, "0.314")

Label(frame_yellow, text="V").grid(row=3, column=0)
Label(frame_yellow, text="≥").grid(row=3, column=1)
yellowV_entry = Entry(frame_yellow, width=8)
yellowV_entry.grid(row=3, column=2, padx=5)
yellowV_entry.insert(0, "0.314")

# ------------------------------
# Process Button
# ------------------------------
Button(root, text="Run Detection",
      bg="#2E8B57", fg="white",
      font=("Arial", 10, "bold"),
      command=process_image).pack(pady=15)
```

```
root.mainloop()
```

Appendix B

Prompts Used from ChatGPT

1. “Explain how HSV color space helps in detecting white and yellow lanes in road images.”
2. “Help me understand how to combine multiple masks to detect lanes in a selected region of interest.”
3. “Teach me how to adjust HSV thresholds and why changing S, V, or H affects lane detection.”
4. “Explain step by step how ROI selection works for focusing lane detection on a specific part of the road.”
5. “Show me how to overlay detected lanes on the original image and why using a red color helps visualize them.”
6. “Help me understand the difference between white mask, yellow mask, and combined mask, and why both are needed for road lanes.”

CLO Mapping

CLO	PLO	Bloom's Taxonomy Level
2	5	P4
3	10	A3

Evaluation Rubric – Digital Image Processing Lab Design Project

	Excellent 1.0	Good 0.7-0.9	Satisfactory 0.5-0.6	Unsatisfactory 0.2-0.4	Poor 0.0-0.1
Viva	Answered all questions correctly	Answered most questions correctly	Answered some questions correctly	Answered very few questions correctly	Answered no questions correctly

	Weight	Excellent 90% - 100%	Good 70% - 89%	Satisfactory 50% - 69%	Unsatisfactory 20% - 49%	Poor < 20%
Data Collection (CLO 2)	**5**	Images data collected of more than 15 images of a particular domain, at-least 2 of which give 'wrong' result.	Images data collected of more than 15 images of a particular domain, more than 2 of which give 'wrong' result.	Images data collected of more than 15 images of a particular domain, more than 4 of which give 'wrong' result.	Images data collected of more than 15 images of a particular domain, more than 6 of which give 'wrong' result.	Images data collected of more than 15 images of a particular domain, more than 8 of which give 'wrong' result.
Algorithm (CLO 2)	**10**	Algorithm developed using classical image processing techniques, that perform the required task for at-least 10 images collected. Block Diagram explains the whole algorithm.	Algorithm developed using classical image processing techniques, that perform the required task for at-least 8 images collected. Block Diagram explains the whole algorithm with minor mistakes.	Algorithm developed using classical image processing techniques, that perform the required task for at-least 6–7 images, block diagram partially explains steps.	Algorithm developed using classical image processing techniques, that perform the required task for 3–5 images, block diagram unclear or missing steps.	Algorithm developed using classical image processing techniques, that perform the required task for images less than 3, block diagram missing or incorrect.
User Interface (CLO 2)	**5**	A graphical user interface with intuitive design, that allows the user to load input	Graphical user interface is mostly user-friendly, works correctly for most	Graphical user interface works for some images, may have unclear	Graphical user interface works for few images, interface is confusing, or outputs	GUI is non-functional or missing, does not allow image loading or parameter selection.

		image, and choose parameters (if required) and displays the output image.	images, minor issues in parameter selection or output display.	navigation or missing features.	are unclear	
Presentation (CLO 3)	**5**	The chosen domain, methodology, algorithm, and results are properly presented and explained, using images, graphs, and block diagrams, and suitable comparisons are made.	The chosen domain, methodology, algorithm, and results are presented with some minor issues, explanations mostly clear using images, graphs, and block diagrams, and comparisons included with some minor issues.	The chosen domain, methodology, algorithm, and results are presented with several issues, explanations somewhat clear using images, graphs, and block diagrams, and comparisons included but with noticeable issues.	The chosen domain, methodology, algorithm, and results are presented with many issues, explanations partially unclear using few images, graphs, or block diagrams, and comparisons are limited or mostly missing.	The chosen domain, methodology, algorithm, and results are barely presented or completely missing, explanations are unclear or absent, images, graphs, and block diagrams are not included, and comparisons are missing.
Report (CLO 3)	**5**	Report includes a complete analysis of the chosen domain, methodology, algorithm, user interface, and results, with proper conclusions drawn. Report follows an appropriate structure, with proper formatting and grammar.	Report includes most of the analysis of the chosen domain, methodology, algorithm, user interface, and results, with some minor issues in conclusions. Structure, formatting, and grammar mostly correct.	Report includes some analysis of the chosen domain, methodology, algorithm, user interface, and results, with several issues in conclusions. Structure, formatting, or grammar partially correct.	Report includes limited analysis, explanations unclear, conclusions missing or mostly incorrect. Structure, formatting, and grammar have many issues.	Report is incomplete or have missing analysis, explanations, conclusions, or proper structure, formatting, and grammar.

***All marks will be scaled based on Viva/ Understanding**